Facultad de Ingeniería y Arquitectura · EP Ingeniería de Sistemas

Lenguaje de Programación II

Guía Práctica — Sesión 11

Seguridad frontend: login, sesión, interceptores, guards, menú por permisos y manejo de 401/403

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	11 · 22 de octubre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Angular 22 · TypeScript · Spring Boot 4 (JWT)	Proyecto base	PharmaSoft (sesiones 7 a 9) + PharmaBackend seguro (sesión 10)
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2

1. Situación de la sesión

Desde la sesión 10, PharmaBackend exige un token en cada petición. La SPA PharmaSoft todavía no lo sabe: al abrir Categorías, Productos o Ventas recibe **401** y no muestra nada. Además, la botica quiere que cada trabajador vea solo lo que le corresponde: el vendedor atiende ventas, el almacenero mantiene el catálogo y la administradora lo ve todo.

Hoy la SPA aprenderá a **iniciar sesión**, guardar la sesión con su vencimiento, **enviar el token** en cada petición, **proteger las rutas**, adaptar el **menú y los botones** a los permisos del usuario y reaccionar correctamente ante un **401** o un **403**.

¿Cómo integras la SPA con un backend seguro para que cada usuario inicie sesión, navegue solo por lo que le corresponde y la aplicación reaccione bien cuando el token vence o falta un permiso?

Reto 01 (actividad práctica, en clase): implementar login, sesión con expiración, interceptores, guards y menú y acciones por permisos en PharmaSoft.

Reto 02 (actividad autónoma): probar la SPA segura con una matriz de casos, avisar antes de que la sesión venza y construir la página «Mi cuenta».

1.1 Resultado de aprendizaje y criterio que se trabaja

Elemento	Descripción
Resultado de aprendizaje de la unidad	Construye una SPA empresarial conectada al backend REST, con layout y navegación, CRUD de tablas independientes y dependientes, formularios cabecera–detalle, consultas y reportes; integra seguridad mediante JWT en el backend y gestión de sesión, guards e interceptores en el frontend.
Criterio 5 del producto	Implementa login, sesión, almacenamiento y expiración del token, guards, interceptores, menú por permisos, rutas protegidas y manejo de 401/403.

1.2 Punto de partida

Requisito	Cómo verificarlo
PharmaBackend con la seguridad de la sesión 10 (rama `feature/seguridad-jwt` o `master`).	En Swagger, `POST /api/v1/auth/login` con `vendedor` / `Vendedor2026*` devuelve un token.
SPA PharmaSoft con Categorías, Productos y Ventas en `develop`.	`ng serve` funciona; hoy las páginas muestran el error 401.
CORS del backend con el origen `http://localhost:4200` y el encabezado `Authorization`.	Ya está en `CorsConfig` desde la sesión 10.

1.3 Ideas clave

Concepto	Qué significa en PharmaSoft
Sesión	Token + vencimiento + datos del usuario (nombre, roles, permisos). La guarda `AuthService` en un signal y en `sessionStorage`.
sessionStorage	Sobrevive a un F5, pero se borra al cerrar el navegador. Adecuado para las computadoras compartidas de la botica.
Interceptor	Función que ve pasar todas las peticiones HTTP. Uno agrega `Authorization: Bearer <token>`; otro reacciona al 401.
Guard	Función que decide si una ruta se abre. `authGuard` exige sesión; `permisoGuard`, el permiso declarado en la ruta.
401 en la SPA	Falta sesión o el token venció: se cierra la sesión y se vuelve al login recordando la página.
403 en la SPA	Hay sesión, pero falta el permiso: se muestra «Acceso denegado» o el mensaje del backend; la sesión continúa.

Usuario	Contraseña	Roles	Ve en el menú
admin	Admin2026*	ADMIN	Inicio, Categorías, Productos, Ventas
vendedor	Vendedor2026*	VENDEDOR	Inicio, Categorías, Productos (solo lectura), Ventas
almacen	Almacen2026*	ALMACENERO	Inicio, Categorías, Productos (con edición)
supervisor	Supervisor2026*	VENDEDOR + ALMACENERO	Todo, con edición del catálogo
inactivo	Inactivo2026*	VENDEDOR (deshabilitado)	No puede entrar

2. Actividad práctica

Volvamos a la botica: mañana cada trabajador entrará con su usuario. Con el Reto 01 conectarás la SPA al backend seguro, paso a paso y con el acompañamiento del docente, y comprobarás que cada usuario ve y puede hacer solo lo que le corresponde.

Campo	Contenido
Título	Seguridad frontend en PharmaSoft: login, sesión, interceptores, guards y menú por permisos
Objetivo	Integra la SPA PharmaSoft con el backend seguro mediante un login que guarda la sesión con su vencimiento, interceptores que envían el token y atienden el 401, guards que protegen las rutas por sesión y por permiso, y un menú y unas acciones que se adaptan a los permisos del usuario, y lo comprueba con los cinco usuarios de prueba.
Duración	100 minutos de trabajo + 20 minutos de pruebas y retroalimentación

Paso 1. Actualizar los repositorios

```
# Backend: con la seguridad de la sesión 10, en ejecución
cd PharmaBackend
git checkout feature/seguridad-jwt     # o master, si ya se fusionó
git pull

# Frontend
cd ../pharma-frontend
git checkout develop
git pull
git checkout -b feature/seguridad-frontend-<apellido>
```

Abre la SPA: Categorías y Productos muestran un error y, en la pestaña **Red** (F12), cada petición responde **401**. Es el punto de partida: la SPA aún no envía el token.

Paso 2. Generar la estructura

```
ng g interface core/auth/auth --type=model
ng g s core/auth/auth-service
ng g interceptor core/interceptors/auth-interceptor --functional
ng g interceptor core/interceptors/error-interceptor --functional
ng g guard core/guards/auth-guard --functional --implements CanActivate
ng g guard core/guards/permiso-guard --functional --implements CanActivate
ng g directive shared/directives/si-permiso
ng g c features/auth/pages/login
ng g c shared/pages/acceso-denegado
```

Si tu versión del CLI no acepta alguna opción, crea el archivo a mano con el nombre indicado: lo importante es el contenido de los pasos siguientes. Elimina los archivos `.spec.ts` que no vayas a usar.

Paso 3. Modelos de la sesión

`LoginResponse` refleja el `LoginResponseDTO` del backend. `Sesion` es lo que guarda la SPA: el token, el **instante de vencimiento** calculado al iniciar sesión y los datos del usuario. `Permiso` es un tipo: si escribes mal un permiso, el compilador lo detecta.

Archivo: `src/app/core/auth/auth.model.ts`

```
/** Permisos que emite PharmaBackend (enum Permiso del backend). */
export type Permiso =
  | 'CATALOGO_LEER'
  | 'CATALOGO_ESCRIBIR'
  | 'CLIENTES_LEER'
  | 'CLIENTES_ESCRIBIR'
  | 'VENTAS_LEER'
  | 'VENTAS_REGISTRAR'
  | 'REPORTES_VER'
  | 'USUARIOS_ADMINISTRAR';

/** Lo que se envía a POST /auth/login. */
export interface LoginRequest {
  username: string;
  password: string;
}

/** Lo que devuelve POST /auth/login (LoginResponseDTO). */
export interface LoginResponse {
  token: string;
  tipo: string;
  expiraEnSegundos: number;
  username: string;
  nombreCompleto: string;
  roles: string[];
  permisos: Permiso[];
}

/** Sesión guardada en el navegador. */
export interface Sesion {
  token: string;
  /** Momento de expiración en milisegundos (Date.now() + expiraEnSegundos * 1000). */
  expiraEn: number;
  username: string;
  nombreCompleto: string;
  roles: string[];
  permisos: Permiso[];
}
```

Paso 4. AuthService: la única fuente de verdad de la sesión

El servicio guarda la sesión en un **signal**, de modo que el menú, el encabezado, los guards y la directiva reaccionan solos cuando cambia. La copia en `sessionStorage` permite recuperarla tras un F5; si la sesión guardada ya venció o está dañada, se descarta. Un temporizador cierra la sesión justo cuando el token vence y lleva al login con el aviso «Su sesión expiró».

Archivo: `src/app/core/auth/auth-service.ts`

```
import { computed, inject, Service, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Router } from '@angular/router';
import { map, Observable, tap } from 'rxjs';
import { environment } from '../../../environments/environment';
import { LoginRequest, LoginResponse, Permiso, Sesion } from './auth.model';

/** Clave con la que se guarda la sesión en sessionStorage. */
const CLAVE_SESION = 'pharmasoft.sesion';

/**
 * Única fuente de verdad sobre la sesión del usuario.
 *
 * - La sesión vive en un signal: el menú, el encabezado y los guards
 *   reaccionan solos cuando cambia.
 * - Se guarda en sessionStorage para sobrevivir a una recarga (F5), pero
 *   se borra al cerrar el navegador: la botica usa computadoras compartidas.
 * - Un temporizador cierra la sesión cuando el token vence.
 */
@Service()
export class AuthService {
  private readonly http = inject(HttpClient);
  private readonly router = inject(Router);
  private readonly url = `${environment.apiUrl}/auth`;
```

```
  private readonly sesionActual = signal<Sesion | null>(this.leerSesionGuardada());
  private temporizador?: ReturnType<typeof setTimeout>;


  readonly sesion = this.sesionActual.asReadonly();
  readonly autenticado = computed(() => this.sesionActual() !== null);


  constructor() {
    const sesion = this.sesionActual();
    if (sesion) this.programarExpiracion(sesion);
  }


  /** Envía las credenciales; si son correctas, guarda la sesión. */
  login(credenciales: LoginRequest): Observable<Sesion> {
    return this.http.post<LoginResponse>(`${this.url}/login`, credenciales).pipe(
      map(r => ({
        token: r.token,
        expiraEn: Date.now() + r.expiraEnSegundos * 1000,
        username: r.username,
        nombreCompleto: r.nombreCompleto,
        roles: r.roles,
        permisos: r.permisos,
      })),
      tap(sesion => this.iniciar(sesion)),
    );
  }


  /**
   * Cierra la sesión y vuelve al login.
   * Si la causa es la expiración, el login lo avisa y, tras entrar de nuevo,
   * regresa a la página donde estaba el usuario.
   */
  cerrarSesion(motivo?: 'expirada', volverA?: string): void {
    clearTimeout(this.temporizador);
    sessionStorage.removeItem(CLAVE_SESION);
    this.sesionActual.set(null);


    const queryParams: Record<string, string> = {};
    if (motivo) queryParams['expirada'] = '1';
    if (volverA && volverA !== '/login') queryParams['returnUrl'] = volverA;
    this.router.navigate(['/login'], { queryParams });
  }


  /** Token vigente o null si no hay sesión o ya venció. */
  token(): string | null {
    const sesion = this.sesionActual();
    return sesion && Date.now() < sesion.expiraEn ? sesion.token : null;
  }


  /** true si el usuario tiene el permiso (lee un signal: es reactivo). */
  tienePermiso(permiso: Permiso): boolean {
    return this.sesionActual()?.permisos.includes(permiso) ?? false;
  }


  private iniciar(sesion: Sesion): void {
    sessionStorage.setItem(CLAVE_SESION, JSON.stringify(sesion));
    this.sesionActual.set(sesion);
    this.programarExpiracion(sesion);
  }


  private programarExpiracion(sesion: Sesion): void {
    clearTimeout(this.temporizador);
    const restante = sesion.expiraEn - Date.now();
    this.temporizador = setTimeout(
      () => this.cerrarSesion('expirada', this.router.url),
      Math.max(restante, 0),
    );
  }


  /** Recupera la sesión tras recargar la página; descarta la vencida o dañada. */
  private leerSesionGuardada(): Sesion | null {
    try {
      const texto = sessionStorage.getItem(CLAVE_SESION);
      if (!texto) return null;
      const sesion = JSON.parse(texto) as Sesion;
      if (!sesion.token || Date.now() >= sesion.expiraEn) {
        sessionStorage.removeItem(CLAVE_SESION);
        return null;
      }
      return sesion;
    } catch {
      sessionStorage.removeItem(CLAVE_SESION);
      return null;
```

```
    }
  }
}
```

¿Por qué sessionStorage y no localStorage?

Ambos son accesibles desde JavaScript (riesgo ante XSS), pero `localStorage` no se borra nunca: en una computadora compartida de la botica, el siguiente trabajador encontraría la sesión abierta. `sessionStorage` se borra al cerrar el navegador y, además, la SPA respeta el vencimiento del token. Una cookie `HttpOnly` sería más segura frente a XSS, pero exige cambios en el backend (cookies y protección CSRF).

Paso 5. Interceptores HTTP

El primer interceptor agrega el token **solo** a las peticiones dirigidas a la API: así nunca se envía a otros servidores. `HttpRequest` es inmutable, por eso se usa `clone`.

Archivo: `src/app/core/interceptors/auth-interceptor.ts`

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { environment } from '../../../environments/environment';
import { AuthService } from '../auth/auth-service';


/**
 * Agrega  Authorization: Bearer <token>  a cada petición dirigida a la API.
 * Los servicios no necesitan saber nada del token.
 */
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = inject(AuthService).token();

  // Sin token, o hacia otro servidor: la petición sale tal cual.
  if (!token || !req.url.startsWith(environment.apiUrl)) {
    return next(req);
  }

  // HttpRequest es inmutable: se envía una copia con el encabezado.
  return next(req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }));
};
```

El segundo reacciona a los errores de seguridad. Ante un **401** cierra la sesión y vuelve al login recordando la página actual, **excepto** en el propio login, donde el 401 significa «credenciales incorrectas». Un **403** no cierra la sesión: el componente muestra el mensaje del backend. En ambos casos el error se vuelve a lanzar para que el componente lo reciba.

Archivo: `src/app/core/interceptors/error-interceptor.ts`

```
import { HttpErrorResponse, HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { Router } from '@angular/router';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../auth/auth-service';

/**
 * Reacción común a los errores de seguridad:
 *
 * - 401: el token falta, venció o no es válido → se cierra la sesión y se
 *   vuelve al login recordando la página actual. Se excluye el propio login:
 *   allí el 401 significa «credenciales incorrectas» y lo muestra el formulario.
 * - 403: el usuario SÍ está autenticado pero no tiene el permiso. No se cierra
 *   la sesión: el componente muestra el mensaje del backend.
 *
 * En ambos casos el error se vuelve a lanzar para que el componente lo reciba.
 */
export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService);
  const router = inject(Router);

  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401 && !req.url.endsWith('/auth/login')) {
        auth.cerrarSesion('expirada', router.url);
      }
      return throwError(() => err);
    }),
  );
};
```

Regístralos en `app.config.ts`. **El orden importa**: primero se agrega el token y luego se atienden los errores.

Archivo: `src/app/app.config.ts`

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './core/interceptors/auth-interceptor';
import { errorInterceptor } from './core/interceptors/error-interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideRouter(routes, withComponentInputBinding()),
    // El orden importa: primero se agrega el token, luego se atienden los errores.
    provideHttpClient(withInterceptors([authInterceptor, errorInterceptor])),
  ],
};
```

Paso 6. Guards

`authGuard` deja pasar solo si hay sesión; si no, devuelve un `UrlTree` hacia el login con la página pedida en `?returnUrl=`. `invitadoGuard` hace lo contrario: impide volver al login si ya se inició sesión.

Archivo: `src/app/core/guards/auth-guard.ts`

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../auth/auth-service';

/**
 * Solo deja pasar si hay sesión. Si no, envía al login y recuerda
 * la URL pedida en ?returnUrl= para volver allí después de entrar.
 */
export const authGuard: CanActivateFn = (_route, state) => {
  const auth = inject(AuthService);
  if (auth.autenticado()) {
    return true;
  }
  return inject(Router).createUrlTree(['/login'], {
    queryParams: { returnUrl: state.url },
  });
};

/** El login solo es para quien NO ha iniciado sesión. */
export const invitadoGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  return auth.autenticado() ? inject(Router).createUrlTree(['/inicio']) : true;
};
```

`permisoGuard` lee el permiso declarado en `data` de la ruta. Un mismo guard sirve para todas las rutas.

Archivo: `src/app/core/guards/permiso-guard.ts`

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { Permiso } from '../auth/auth.model';
import { AuthService } from '../auth/auth-service';

/**
 * Exige el permiso declarado en la ruta:  data: { permiso: 'VENTAS_LEER' }.
 * Si falta, muestra la página de acceso denegado (no el login: el usuario
 * ya está identificado).
 *
 * Es una ayuda para la experiencia de usuario: la protección real la hace
 * el backend, que responde 403 aunque alguien modifique el JavaScript.
 */
export const permisoGuard: CanActivateFn = route => {
  const permiso = route.data['permiso'] as Permiso | undefined;
  if (!permiso || inject(AuthService).tienePermiso(permiso)) {
    return true;
  }
  return inject(Router).createUrlTree(['/acceso-denegado']);
};
```

Paso 7. Pantalla de login

El login recibe `returnUrl` y `expirada` desde la URL como `input()` (gracias a `withComponentInputBinding`, configurado en la sesión 7). Tras iniciar sesión vuelve a la página pedida, pero **solo si es una ruta interna**: así nadie puede usar el login para enviar al usuario a un sitio falso (redirección abierta).

Archivo: `src/app/features/auth/pages/login/login.ts`

```
import { Component, inject, input, signal } from '@angular/core';
import { HttpErrorResponse } from '@angular/common/http';
import { NonNullableFormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../../../core/auth/auth-service';
import { mensajeError } from '../../../../core/utils/http-error';

@Component({
  selector: 'app-login',
  imports: [ReactiveFormsModule],
```

```
  templateUrl: './login.html',
  styleUrl: './login.css',
})
export class Login {
  private readonly fb = inject(NonNullableFormBuilder);
  private readonly auth = inject(AuthService);
  private readonly router = inject(Router);

  /** ?returnUrl=/ventas  → página a la que se vuelve después de entrar. */
  readonly returnUrl = input<string>();
  /** ?expirada=1  → la sesión anterior venció. */
  readonly expirada = input<string>();

  protected readonly enviando = signal(false);
  protected readonly error = signal<string | null>(null);
  protected readonly verClave = signal(false);

  protected readonly form = this.fb.group({
    username: ['', [Validators.required, Validators.maxLength(50)]],
    password: ['', [Validators.required, Validators.maxLength(72)]],
  });

  ingresar(): void {
    if (this.form.invalid) {
      this.form.markAllAsTouched();
      return;
    }
    this.enviando.set(true);
    this.error.set(null);

    this.auth.login(this.form.getRawValue()).subscribe({
      next: () => this.router.navigateByUrl(this.destino()),
      error: (err: HttpErrorResponse) => {
        this.enviando.set(false);
        this.form.controls.password.reset();
        this.error.set(mensajeError(err));
      },
    });
  }

  /**
   * Solo se aceptan rutas internas de la SPA ("/ventas"), nunca direcciones
   * externas ("https://…" o "//sitio"): evita una redirección abierta.
   */
  private destino(): string {
    const url = this.returnUrl();
    return url && url.startsWith('/') && !url.startsWith('//') ? url : '/inicio';
  }
}
```

Archivo: `src/app/features/auth/pages/login/login.html`

```
<main class="fondo">
  <section class="card login" aria-labelledby="titulo-login">
    <p class="marca">PharmaSoft</p>
    <h1 id="titulo-login">Iniciar sesión</h1>

    @if (expirada()) {
      <div class="aviso" role="status">Su sesión expiró. Inicie sesión nuevamente.</div>
    }
    @if (error()) {
      <div class="alerta" role="alert">{{ error() }}</div>
    }

    <form [formGroup]="form" (ngSubmit)="ingresar()" novalidate>
      <label>
        Usuario
        <input class="input" formControlName="username" autocomplete="username" autofocus />
      </label>
      @if (form.controls.username.touched && form.controls.username.invalid) {
        <small class="error">Ingrese su usuario.</small>
      }

      <label>
        Contraseña
        <input class="input" [type]="verClave() ? 'text' : 'password'"
               formControlName="password" autocomplete="current-password" />
      </label>
```

```
      @if (form.controls.password.touched && form.controls.password.invalid) {
        <small class="error">Ingrese su contraseña.</small>
      }

      <label class="ver">
        <input type="checkbox" [checked]="verClave()" (change)="verClave.set(!verClave())" />
        Mostrar contraseña
      </label>

      <button type="submit" class="btn btn-primary" [disabled]="enviando()">
        {{ enviando() ? 'Ingresando…' : 'Ingresar' }}
      </button>
    </form>
  </section>
</main>
```

Archivo: `src/app/features/auth/pages/login/login.css`

```
.fondo {
  min-height: 100vh; display: grid; place-items: center; padding: 16px;
  background: linear-gradient(135deg, #0f2e5c 0%, #1f5aa6 100%);
}
.login { width: 100%; max-width: 400px; }
.marca { margin: 0; color: #f2a900; font-weight: 800; font-size: 22px; }
h1 { margin: 4px 0 16px; font-size: 24px; color: #0f2e5c; }
form { display: flex; flex-direction: column; gap: 10px; }
label { display: flex; flex-direction: column; gap: 4px; font-weight: 600; font-size: 14px; }
.ver { flex-direction: row; align-items: center; gap: 8px; font-weight: 400; }
.error { color: #b42318; margin-top: -6px; }
.aviso { padding: 10px 14px; margin-bottom: 12px; border-radius: 8px; background: #fff6dc; color: #7a5200;
}
button { margin-top: 8px; padding: 10px; }
```

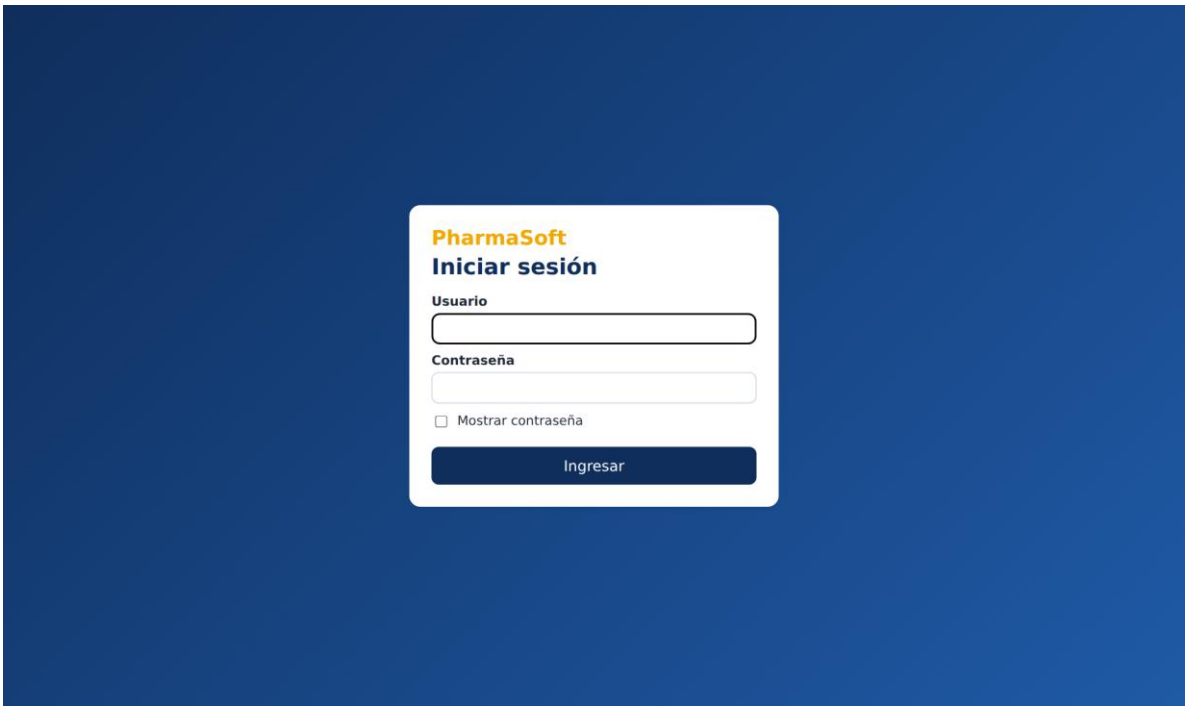


Figura 1. Pantalla de login fuera del layout. Nota. Elaboración propia.

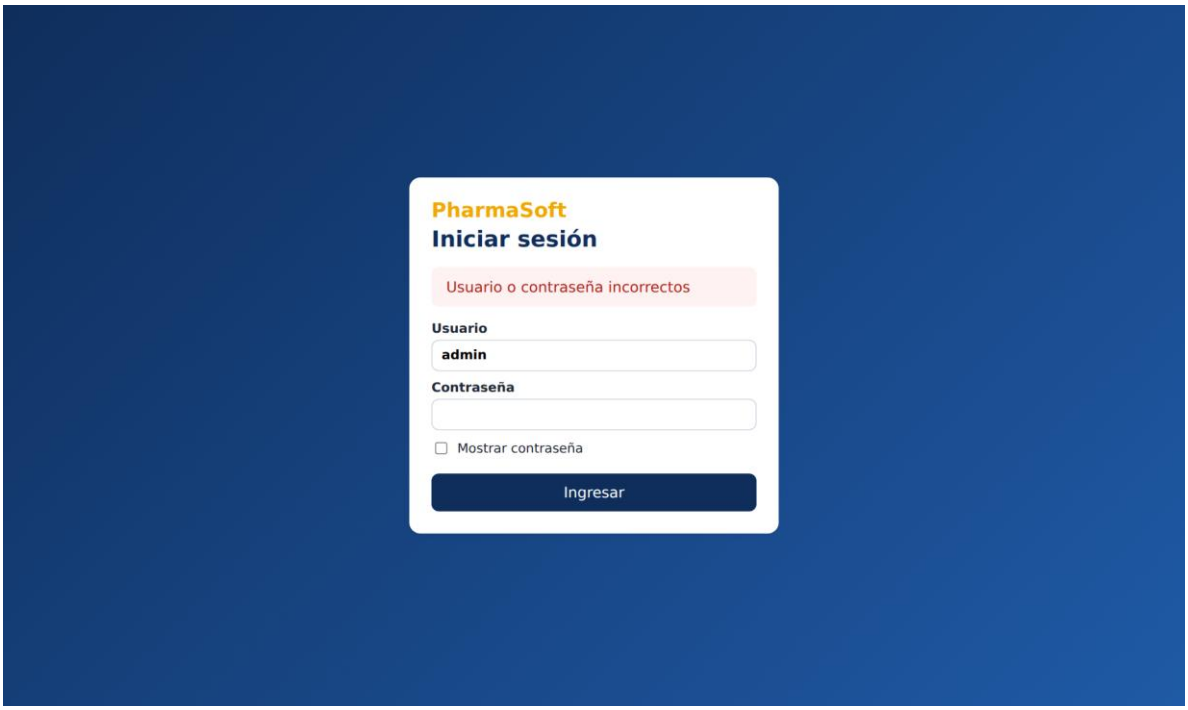


Figura 2. Credenciales incorrectas: 401 del backend mostrado en el formulario. Nota. Elaboración propia.

Paso 8. Rutas protegidas

El login queda fuera del layout. Todo lo que cuelga de `MainLayout` exige sesión: `canActivate` al entrar al layout y `canActivateChild` en cada navegación interna. Cada módulo declara el permiso que necesita.

Archivo: `src/app/app.routes.ts`

```
import { Routes } from '@angular/router';
import { MainLayout } from './layout/main-layout/main-layout';
import { authGuard, invitadoGuard } from './core/guards/auth-guard';
import { permisoGuard } from './core/guards/permiso-guard';


export const routes: Routes = [
  // Pública: fuera del layout y solo para quien no inició sesión
  {
    path: 'login',
    title: 'Iniciar sesión',
    canActivate: [invitadoGuard],
    loadComponent: () => import('./features/auth/pages/login/login').then(m => m.Login),
  },
  // Privadas: todo lo que cuelga del layout exige sesión
  {
    path: '',
    component: MainLayout,
    canActivate: [authGuard],
    canActivateChild: [authGuard],
    children: [
      { path: '', pathMatch: 'full', redirectTo: 'inicio' },
      {
        path: 'inicio',
        title: 'Inicio',
        loadComponent: () => import('./features/inicio/inicio').then(m => m.Inicio),
      },
      {
        path: 'categorias',
        canActivate: [permisoGuard],
        data: { permiso: 'CATALOGO_LEER' },
        loadChildren: () =>
          import('./features/categorias/categorias.routes').then(m => m.CATEGORIAS_ROUTES),
      },
      {
        path: 'productos',
        canActivate: [permisoGuard],
        data: { permiso: 'CATALOGO_LEER' },
        loadChildren: () =>
          import('./features/productos/productos.routes').then(m => m.PRODUCTOS_ROUTES),
      },
      {
```

```
        path: 'ventas',
        canActivate: [permisoGuard],
        data: { permiso: 'VENTAS_LEER' },
        loadChildren: () =>
          import('./features/ventas/ventas.routes').then(m => m.VENTAS_ROUTES),
      },
      {
        path: 'acceso-denegado',
        title: 'Acceso denegado',
        loadComponent: () =>
          import('./shared/pages/acceso-denegado/acceso-denegado').then(m => m.AccesoDenegado),
      },
    ],
  },
  {
    path: '**',
    title: 'Página no encontrada',
    loadComponent: () =>
      import('./shared/pages/no-encontrado/no-encontrado').then(m => m.NoEncontrado),
  },
];
```

Las rutas que crean o editan declaran el permiso de escritura. Haz lo mismo en `categorias.routes.ts` y `productos.routes.ts` (con `CATALOGO_ESCRIBIR`):

Archivo: `src/app/features/ventas/ventas.routes.ts`

```
import { Routes } from '@angular/router';
import { permisoGuard } from '../../core/guards/permiso-guard';
import { VentaList } from './pages/venta-list/venta-list';
import { VentaForm } from './pages/venta-form/venta-form';
import { VentaDetalle } from './pages/venta-detalle/venta-detalle';

export const VENTAS_ROUTES: Routes = [
  { path: '', component: VentaList, title: 'Consulta de ventas' },
  {
    path: 'nueva', component: VentaForm, title: 'Nueva venta',
    canActivate: [permisoGuard], data: { permiso: 'VENTAS_REGISTRAR' },
  },
  { path: ':id', component: VentaDetalle, title: 'Detalle de venta' },
];
```

Archivo: `src/app/features/productos/productos.routes.ts`

```
import { Routes } from '@angular/router';
import { permisoGuard } from '../../core/guards/permiso-guard';
import { ProductoList } from './pages/producto-list/producto-list';
import { ProductoForm } from './pages/producto-form/producto-form';

export const PRODUCTOS_ROUTES: Routes = [
  { path: '', component: ProductoList, title: 'Productos' },
  {
    path: 'nuevo', component: ProductoForm, title: 'Nuevo producto',
    canActivate: [permisoGuard], data: { permiso: 'CATALOGO_ESCRIBIR' },
  },
  {
    path: ':id/editar', component: ProductoForm, title: 'Editar producto',
    canActivate: [permisoGuard], data: { permiso: 'CATALOGO_ESCRIBIR' },
  },
];
```

Y la página que se muestra cuando falta un permiso:

Archivo: `src/app/shared/pages/acceso-denegado/acceso-denegado.ts`

```
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';


@Component({
  selector: 'app-acceso-denegado',
  imports: [RouterLink],
  template: `
    <section class="card">
      <h2>403 · Acceso denegado</h2>
      <p>Su usuario no tiene permiso para abrir esta página. Si lo necesita, solicítelo al
administrador.</p>
      <a routerLink="/inicio" class="btn btn-primary">Volver al inicio</a>
    </section>
  `,
})
export class AccesoDenegado {}
```

Paso 9. Menú y encabezado según la sesión

Cada opción del menú declara su permiso. El sidebar filtra con un `computed`, que se recalcula si la sesión cambia.

Archivo: `src/app/core/config/menu.ts`

```
import { Permiso } from '../auth/auth.model';

export interface MenuItem {
  etiqueta: string;
  ruta: string;
  icono: string;
  /** Permiso necesario para ver la opción; sin permiso = visible para todos. */
  permiso?: Permiso;
}

export const MENU: MenuItem[] = [
  { etiqueta: 'Inicio', ruta: '/inicio', icono: '🏠' },
  { etiqueta: 'Categorías', ruta: '/categorias', icono: '🗂', permiso: 'CATALOGO_LEER' },
  { etiqueta: 'Productos', ruta: '/productos', icono: '🏷', permiso: 'CATALOGO_LEER' },
  { etiqueta: 'Ventas', ruta: '/ventas', icono: '🧾', permiso: 'VENTAS_LEER' },
];
```

Archivo: `src/app/layout/sidebar/sidebar.ts`

```
import { Component, computed, inject } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { MENU } from '../../core/config/menu';
import { AuthService } from '../../core/auth/auth-service';


@Component({
  selector: 'app-sidebar',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './sidebar.html',
  styleUrl: './sidebar.css',
})
export class Sidebar {
  private readonly auth = inject(AuthService);


  /** Solo las opciones que el usuario puede abrir. Se recalcula si cambia la sesión. */
  protected readonly menu = computed(() =>
    MENU.filter(item => !item.permiso || this.auth.tienePermiso(item.permiso)));
}
```

En `sidebar.html` cambia `menu` por `menu()`. El encabezado muestra el nombre, los roles, la hora de expiración y el botón «Cerrar sesión»:

Archivo: `src/app/layout/header/header.ts`

```
import { Component, inject, output } from '@angular/core';
import { DatePipe } from '@angular/common';
import { RouterLink } from '@angular/router';
import { AuthService } from '../../core/auth/auth-service';


@Component({
  selector: 'app-header',
  imports: [RouterLink, DatePipe],
  templateUrl: './header.html',
  styleUrl: './header.css',
})
export class Header {
  protected readonly auth = inject(AuthService);
  readonly toggleMenu = output<void>();


  salir(): void {
    this.auth.cerrarSesion();
  }
}
```

Archivo: `src/app/layout/header/header.html`

```
<header class="header">
  <button type="button" class="btn-menu" aria-label="Mostrar u ocultar menú"
(click)="toggleMenu.emit()">☰</button>
  <a routerLink="/inicio" class="marca">PharmaSoft</a>
  <nav class="menubar">
    <a routerLink="/inicio">Inicio</a>
    <a href="http://localhost:8080/swagger-ui.html" target="_blank" rel="noopener">API</a>
  </nav>
  @if (auth.sesion(); as s) {
    <div class="usuario">
      <span class="nombre">{{ s.nombreCompleto }}</span>
      <span class="detalle">{{ s.roles.join(' · ') }} · expira {{ s.expiraEn | date: 'HH:mm' }}</span>
    </div>
    <button type="button" class="btn-salir" (click)="salir()">Cerrar sesión</button>
  }
</header>
```

En `header.css` reemplaza la regla `.usuario` por estas:

```
.usuario { margin-left: auto; display: flex; flex-direction: column; align-items: flex-end; line-height:
1.2; }
.nombre { font-weight: 600; font-size: 14px; }
.detalle { font-size: 12px; color: #cadcfc; }
.btn-salir {
  padding: 6px 12px; border: 1px solid #cadcfc; border-radius: 8px; background: none;
  color: #fff; font: inherit; font-size: 14px; cursor: pointer;
}
.btn-salir:hover { background: rgb(255 255 255 / 0.12); }
```

Paso 10. Botones según el permiso

La directiva estructural `*appSiPermiso` muestra un elemento solo si el usuario tiene el permiso. Como `tienePermiso()` lee un signal, el `effect` vuelve a evaluarse si la sesión cambia.

Archivo: `src/app/shared/directives/si-permiso.ts`

```
import { Directive, effect, inject, input, TemplateRef, ViewContainerRef } from '@angular/core';
import { Permiso } from '../../core/auth/auth.model';
import { AuthService } from '../../core/auth/auth-service';


/**
 * Muestra el elemento solo si el usuario tiene el permiso:
 *
 *   <a *appSiPermiso="'CATALOGO_ESCRIBIR'" routerLink="nuevo">+ Nuevo</a>
 *
 * Como tienePermiso() lee un signal, el effect vuelve a evaluarse si la
 * sesión cambia (por ejemplo, al cerrar sesión).
 */
@Directive({ selector: '[appSiPermiso]' })
export class SiPermiso {
  private readonly auth = inject(AuthService);
  private readonly plantilla = inject(TemplateRef);
  private readonly contenedor = inject(ViewContainerRef);
  private visible = false;


  readonly appSiPermiso = input.required<Permiso>();


  constructor() {
    effect(() => {
      const permitido = this.auth.tienePermiso(this.appSiPermiso());
      if (permitido && !this.visible) {
        this.contenedor.createEmbeddedView(this.plantilla);
        this.visible = true;
      } else if (!permitido && this.visible) {
        this.contenedor.clear();
        this.visible = false;
      }
    });
  }
}
```

Úsala en las listas (agrega `SiPermiso` a los `imports` del componente). En `producto-list.html` y `categoria-list.html`:

```
<a *appSiPermiso="'CATALOGO_ESCRIBIR'" routerLink="nuevo" class="btn btn-primary">+ Nuevo producto</a>
...
<th *appSiPermiso="'CATALOGO_ESCRIBIR'">Acciones</th>
...
<td *appSiPermiso="'CATALOGO_ESCRIBIR'" class="acciones"> ... </td>
```

Y en `venta-list.html`, para «+ Nueva venta»:

```
<a *appSiPermiso="'VENTAS_REGISTRAR'" routerLink="nueva" class="btn btn-primary">+ Nueva venta</a>
```

Ocultar no es proteger

Los guards, el menú y la directiva mejoran la experiencia, pero cualquiera puede modificar el JavaScript en su navegador. La protección real está en el backend: aunque alguien llegue a un botón oculto, PharmaBackend responde 403.

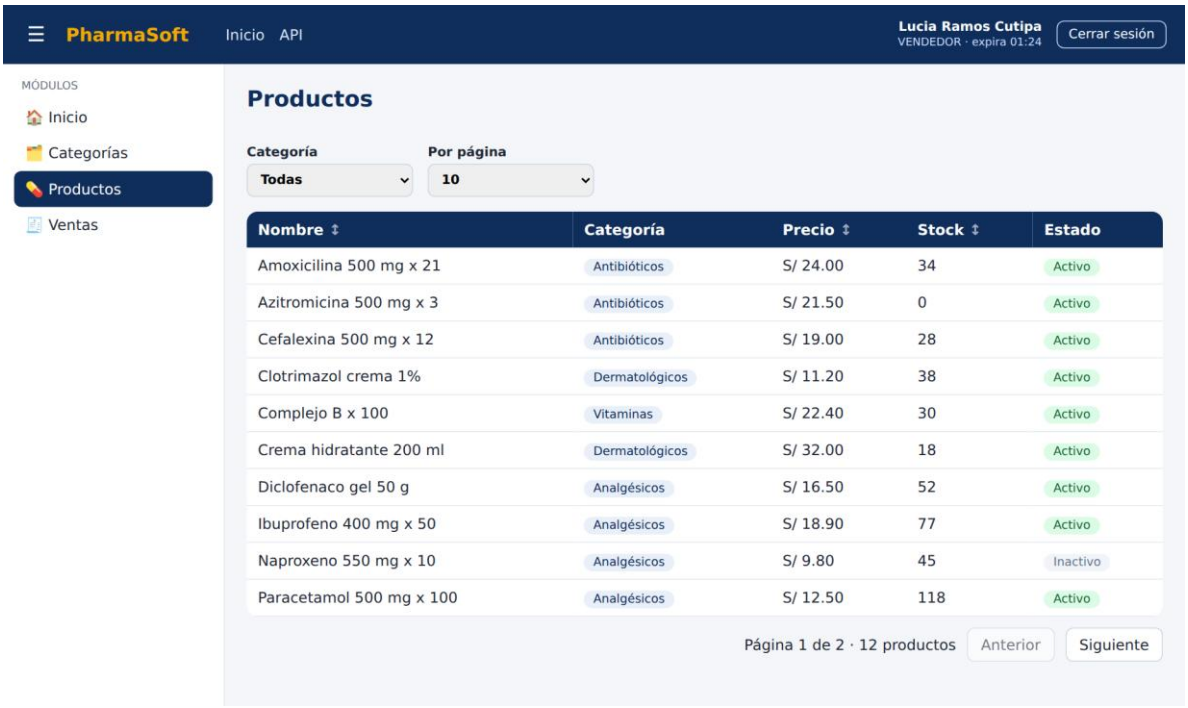


Figura 3. VENDEDOR: ve Ventas en el menú y Productos sin botones de edición. Nota. Elaboración propia.

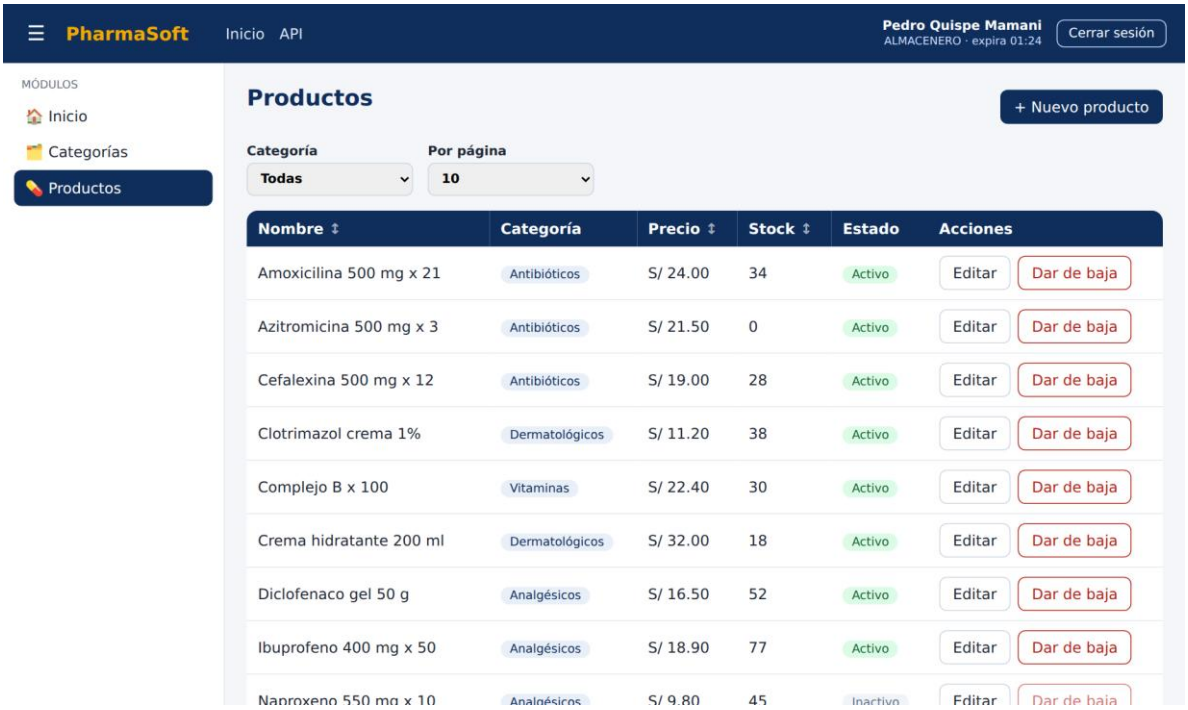


Figura 4. ALMACENERO: sin Ventas en el menú y con «Nuevo producto», «Editar» y «Dar de baja». Nota. Elaboración propia.

Paso 11. Probar la SPA segura

Con PharmaBackend y `ng serve` en ejecución, abre la pestaña **Red** (F12) y la pestaña **Aplicación → Almacenamiento de sesión**. Ejecuta las pruebas en orden y guarda una captura de cada una.

N.º	Acción	Resultado esperado
1	Sin sesión, escribir `http://localhost:4200/ventas`	Login con `?returnUrl=%2Fventas`.
2	Pulsar «Ingresar» con el formulario vacío	Mensajes en ambos campos; ninguna petición.

N.º	Acción	Resultado esperado
3	Login `admin` con contraseña incorrecta	«Usuario o contraseña incorrectos» (401); sigue en el login.
4	Login `inactivo` / `Inactivo2026*`	El mismo mensaje: no revela que la cuenta existe.
5	Login `vendedor` / `Vendedor2026*`	Vuelve a /ventas; el encabezado muestra nombre, rol y hora de expiración.
6	Revisar en Red un `GET /productos`	Lleva `Authorization: Bearer eyJ…`; el login no lo lleva.
7	Revisar Almacenamiento de sesión	Clave `pharmasoft.sesion` con token, `expiraEn` y permisos.
8	Pulsar F5 en Productos	Sigue con sesión en la misma página.
9	VENDEDOR en Productos	Sin «Nuevo producto» ni columna Acciones.
10	VENDEDOR escribe `/productos/nuevo`	Página «403 · Acceso denegado» (permisoGuard).
11	Con sesión, escribir `/login`	Redirige a /inicio (invitadoGuard).
12	«Cerrar sesión»	Login; `pharmasoft.sesion` ya no existe.
13	Login `almacen`; escribir `/ventas`	Ventas no está en el menú y la URL directa muestra «Acceso denegado».
14	Login `supervisor`	Ve Ventas y también los botones de edición del catálogo (unión de permisos).
15	En Almacenamiento de sesión, cambiar un carácter del token y pulsar F5	El backend responde 401; la SPA vuelve al login con «Su sesión expiró».
16	En el backend, `app.jwt.expiracion-minutos: 1`; iniciar sesión y esperar	Al minuto, la SPA vuelve al login con «Su sesión expiró»; al entrar, regresa a la misma página. Restaura el valor 60.

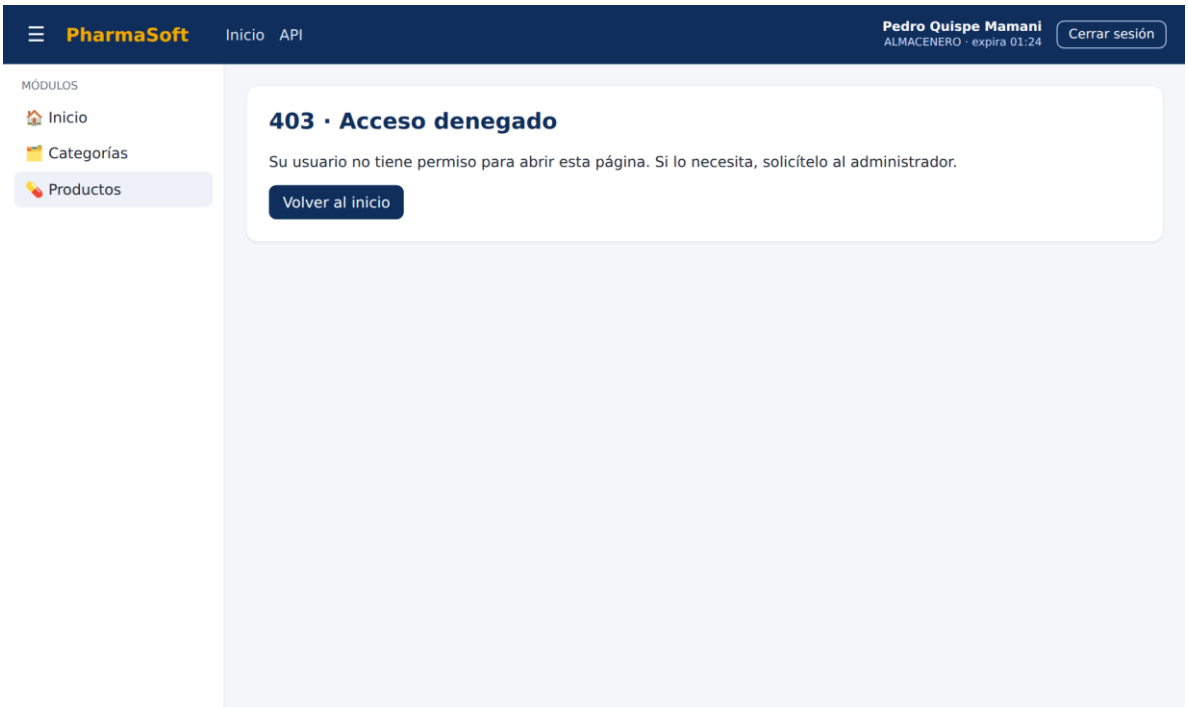


Figura 5. ALMACENERO intenta abrir /ventas: página de acceso denegado. Nota. Elaboración propia.

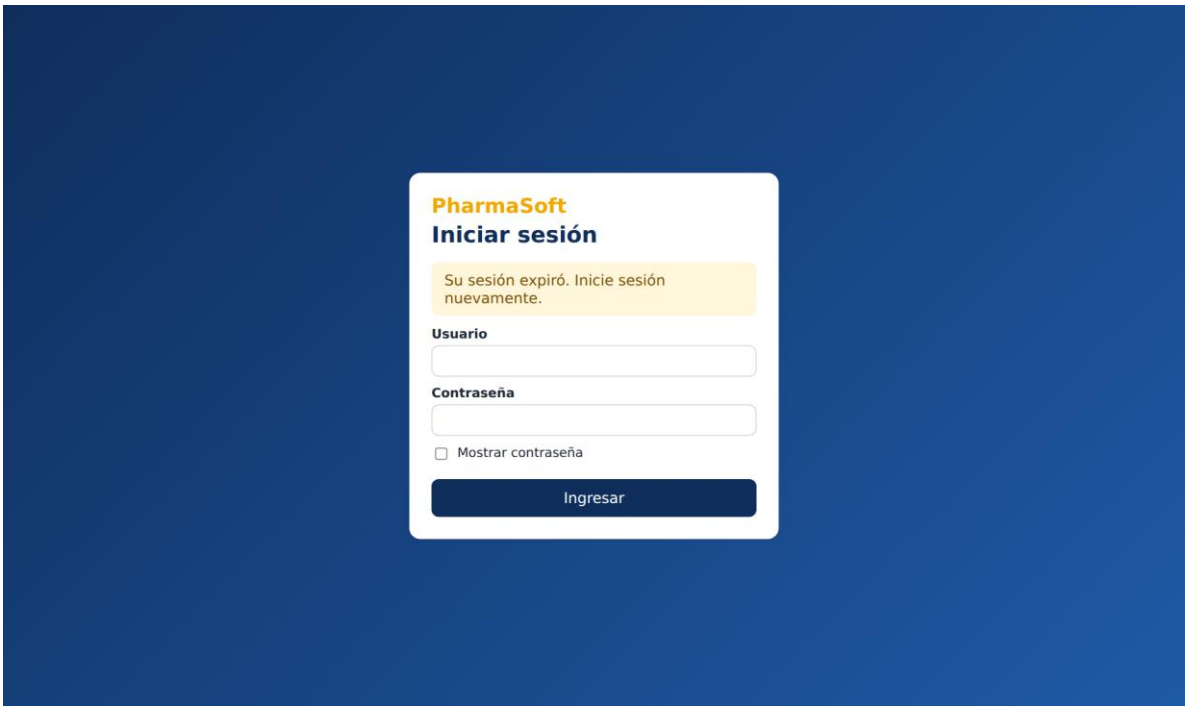


Figura 6. El token venció: la SPA vuelve al login con el aviso. Nota. Elaboración propia.

Paso 12. Versionar el trabajo

```
git add .
git commit -m "feat(auth): AuthService con sesión en sessionStorage y expiración"
git commit -am "feat(auth): interceptores de token y de errores 401"
git commit -am "feat(auth): login, guards y rutas protegidas"
git commit -am "feat(auth): menú y acciones según permisos"
git push -u origin feature/seguridad-frontend-<apellido>
```

Haz cada commit cuando esa parte funcione y abre una solicitud de incorporación hacia `develop`. Después de crear archivos nuevos usa `git add .` antes del commit.

Producto esperado

- Rama `feature/seguridad-frontend-<apellido>` con al menos cuatro commits que sigan la convención y una solicitud de incorporación hacia `develop`.
- SPA PharmaSoft con login, sesión con expiración, interceptores, guards por sesión y por permiso, y menú y botones según los permisos.
- Documento PDF `Apellido_LP2_S11_Practica.pdf` con las capturas de las 16 pruebas del paso 11.

Lista de cotejo

N.º	Criterio observable	Sí	No
1	El login valida el formulario, muestra el 401 del backend y no revela qué dato falló.		
2	`AuthService` guarda la sesión en un signal y en `sessionStorage`, con el instante de vencimiento.		
3	La sesión sobrevive a un F5 y se descarta si venció; un temporizador la cierra al vencer.		

N.º	Criterio observable	Sí	No
4	`authInterceptor` agrega `Authorization: Bearer` solo a las peticiones dirigidas a la API.		
5	`errorInterceptor` cierra la sesión ante un 401 (salvo en el login) y no ante un 403.		
6	`authGuard` protege el layout y recuerda la página pedida en `returnUrl`; `invitadoGuard` protege el login.		
7	`permisoGuard` usa el permiso declarado en `data` y lleva a «Acceso denegado».		
8	El login solo redirige a rutas internas (sin redirección abierta).		
9	El menú y los botones se muestran según los permisos (computed y `*appSiPermiso`).		
10	El encabezado muestra el usuario, sus roles y la expiración, y permite cerrar sesión.		
11	La rama tiene al menos cuatro commits según la convención y una solicitud de incorporación hacia `develop`.		

3. Problemas frecuentes

Síntoma	Causa probable	Solución
Después del login, todo sigue respondiendo 401	Los interceptores no están registrados o falta `withInterceptors`.	Revisa `app.config.ts` (paso 5).
Error de CORS solo en las peticiones con token	El backend no permite el encabezado `Authorization`.	Agrégalo a `allowedHeaders` en `CorsConfig` (ya incluido en la sesión 10).
Con contraseña incorrecta, la página se recarga y no muestra el mensaje	El `errorInterceptor` también redirige el 401 del login.	Excluye la URL `/auth/login` (paso 5).
Tras un F5 se pierde la sesión	La sesión solo vive en el signal.	Guárdala en `sessionStorage` y léela al crear el servicio (`leerSesionGuardada`).
El menú no cambia al cambiar de usuario	El menú es un arreglo fijo, no un `computed`.	Usa `computed` y, en la plantilla, `menu()`.
Al escribir la URL se abre una página sin permiso	Solo se ocultó el botón.	Agrega `canActivate: [permisoGuard]` y `data: { permiso }` a la ruta.
Bucle de redirecciones entre /login y /inicio	`authGuard` aplicado también a la ruta del login.	El login usa `invitadoGuard`; `authGuard` va en el layout.
«NG0203: inject() must be called from an injection context»	Se llamó a `inject()` dentro de un callback (por ejemplo, en `catchError`).	Llama a `inject()` al inicio del interceptor o guard, fuera de los callbacks.

4. Referencias

Angular. (s. f.-a). *Interceptors*. https://angular.dev/guide/http/interceptors

Angular. (s. f.-b). *Route guards*. https://angular.dev/guide/routing/route-guards

Angular. (s. f.-c). *Security*. https://angular.dev/best-practices/security

OWASP Foundation. (s. f.-a). *HTML5 security cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/HTML5*Security*Cheat_Sheet.html

OWASP Foundation. (s. f.-b). *Unvalidated redirects and forwards cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated*Redirects*and*Forwards*Cheat_Sheet.html